

DataLeakDetector h 分支技术工作报告

生成日期: 2026-06-29

分支: h / origin/h

基线: main at 459753f

最新提交: 87f8aa7 Add offline detection benchmark

工作概览

本轮 h 分支主要围绕“降低 VLM 调用成本、提升真实外发场景召回、强化 Qwen/VLM 响应鲁棒性、建立可复用离线评估”展开。相对 main, 共新增 8 个提交, 涉及 10 个文件, 代码与 fixture 合计 2721 insertions / 50 deletions。

从结果上看, 这次工作不是单点修 bug, 而是把数据泄露检测链路从“尽量让 VLM 看画面判断”调整为“先用日志做便宜且确定的判断, 再让 VLM 处理日志说不清的画面语义”。这样做的直接收益有两个: 一是减少 VLM 图片调用, 降低 token 成本和响应时间; 二是补齐真实办公中常见但不一定表现为 file_upload 的外发方式, 例如云盘同步、U 盘拷贝、HTTP POST、截图录屏、屏幕共享等。

为了避免检测能力只停留在“代码改了但不知道效果如何”, 本轮还同步建设了 fixture 和离线 benchmark。fixture 用来保存典型正负样例, benchmark 用来复跑这些样例并输出 precision、recall、F1、失败样例和 VLM fallback 次数。后续只要继续补充真实标注样例, 就可以持续观察检测规则是否退化。

VLM 后处理

12/12

fixture 全通过; 无 FP/FN

日志分流

16/16

log-first 与 fallback 期望一致

离线 VLM 压力

4 calls / 24 frames

按每次最多 6 帧估算

核心成果:

- 增加 log-first 确定性检测: 日志证据足够时直接产出上传/告警事件, 避免不必要的 VLM 调用。
- 增加 VLM fallback gate: 只有敏感上下文附近出现 AI、剪贴板、截图、录屏、云同步等模糊外发信号时才触发 VLM。
- 强化 Qwen/VLM 返回解析: 支持 JSON fence、前置说明、裸数组、字段缺失、重复事件、正常阅读噪声等情况。
- 扩展真实风险覆盖: 重命名、压缩、浏览器 temp 上传、截图/录屏、剪贴板、云同步、U 盘、HTTP POST、延迟导出再上传。
- 新增离线 benchmark: 不调用线上 VLM, 基于 fixture 量化准确率、召回率与 VLM 触发压力, 便于后续接真实标注集。

项目流程与模块职责

DataLeakDetector 的目标是从系统日志和录屏中还原“敏感数据是否离开可信环境”。它不是只看一个文件是否被打开, 而是要判断“敏感文件经过哪些操作, 最终有没有被发送、上传、同步、截图、录屏或以其他方式外发”。

整体流程可以理解为一个证据链:

敏感源文件

- > 打开/复制/重命名/压缩/导出
- > 上传/同步/粘贴/截图/录屏/屏幕共享
- > 形成告警事件
- > 注入 Datalog 推理
- > 输出可解释泄露路径

模块	输入	职责与输出
ScreenMonitor	操作系统层面的文件、窗口、剪贴板、截图等日志, 以及录屏视频。	提供原始证据来源。例如记录某个文件被打开、某个浏览器窗口发生上传、用户切换到 ChatGPT 页面等。

RiskHunter / 模块 3	日志、敏感文件配置、黑白名单配置。	本轮重点增强。先用 LogFirstDetector 尝试直接发现上传链路；如果证据不足，再决定是否调用后续视觉分析。
FileTracker / 模块 2	当前敏感文件事件、模块 1 输出的行为事件。	追踪文件派生关系，例如敏感 Word 导出 PDF、Excel 压缩成 zip、一个文件拆成多个文件等。
FrameAnalyzer 模块 1	录屏视频、目标关键词、搜索时间范围。	OCR 筛选关键帧，控制送给 VLM 的帧数，并把 VLM 返回结果解析成结构化行为事件。
ThreatDetector 模块 4	模块 3 的上传事件、文件映射、操作记录。	将事件转成 Datalog fact，推理完整泄露路径，例如 OpenFile -> TransferFile -> LeakFile。

本轮工作主要集中在模块 1、模块 3、run_e2e.py 和测试/评估体系上。模块 1 负责让 VLM 输入输出更可靠，模块 3 负责让日志能够先行解决问题，run_e2e.py 负责决定什么时候跳过 VLM、什么时候进入 VLM，benchmark 负责量化效果。

业务问题背景

原有链路更依赖视频帧与 VLM 判断。该方案在演示场景中可以工作，但放到真实办公环境会遇到三个问题。

问题	真实表现	本轮处理方式
成本问题	如果每个可疑时间段都送 VLM，长录屏、多窗口办公、频繁复制粘贴都会带来大量图片 token。	先用日志判断。日志能证明上传链路时直接返回结果；只有日志无法判断内容时才调用 VLM。
召回问题	真实外发不一定叫“上传”：可能是复制到 Dropbox、拷到 U 盘、curl POST、导出 PDF 后再发邮件。	把外发抽象成“敏感数据离开可信本地环境”，补充云同步、可移动介质、网络 POST、派生文件映射。
鲁棒性问题	VLM 输出字段不稳定，可能返回 operation 而不是 operation_type，也可能把“薪资”写成“薪酬”。	做字段别名归一化、敏感概念组、风险动作词、低价值正常事件过滤。

通俗理解：本轮不是把系统改成“看到敏感文件就报警”，而是把检测链路拆成三层判断。

- 第一层：日志证据是否已经足够。例如“敏感文件 -> 压缩包 -> 邮件附件上传”，日志本身可以给出结论。
- 第二层：日志是否提示需要看画面。例如“敏感文件打开后 20 秒出现 ChatGPT 粘贴”，日志知道动作可疑，但不知道粘贴内容。
- 第三层：VLM 输出是否可信、是否相关。例如模型返回了多个事件，要保留外发动作，丢掉正常阅读和重复描述。

总体方案

整体链路从“直接依赖 VLM”调整为“日志优先、视觉兜底、结果可评估”。

阶段	输入与判断	输出
Log-first	输入系统日志，识别敏感文件打开、文件派生、上传/同步/POST/U 盘等确定性链路。	如果链路完整，直接输出 UploadEvent、告警等级、文件映射链；此时跳过 VLM。
Fallback gate	当没有确定上传事件时，检查敏感上下文附近是否有 AI、剪贴板、截图、录屏等模糊外发信号。	决定是否值得调用 VLM。无敏感上下文或普通聊天噪声会直接跳过。
FrameAnalyzer	对 OCR 命中的关键帧做采样、压缩并送 VLM；解析模型返回事件。	输出经过过滤和去重的行为事件，例如粘贴外发、附件上传、屏幕共享、录屏等。

ThreatDetector	将 log-first/module3 结果注入 Datalog fact, 连接 OpenFile、TransferFile、CrossProcessTransfer、LeakFile。	输出可解释的数据泄露路径, 而不仅是一条孤立告警。
Benchmark	使用 fixture 离线复跑 VLM 后处理、日志分流和确定性解决率。	输出 precision、recall、F1、case failures、估算 VLM 调用次数。

成本控制的关键点是：VLM 不再是入口，而是兜底判断器。确定性日志可以解决的场景不进入 VLM；没有敏感上下文的负例也不进入 VLM；只有“敏感上下文 + 模糊外发动作 + 时间邻近”的情况才进入 VLM。

完整检测链路示例

以下示例展示一次典型外发从原始日志到最终报告的处理过程。

输入日志：

```
[
  {
    "event_type": "file_open",
    "file_path": "C:/Users/alice/Desktop/员工薪资明细表Q4.xlsx",
    "process_info": {"process_name": "excel.exe"}
  },
  {
    "event_type": "compressed",
    "file_path": "C:/Users/alice/Desktop/员工薪资明细表Q4_供应商.zip",
    "process_info": {"process_name": "7z.exe"}
  },
  {
    "event_type": "file_upload",
    "file_path": "C:/Users/alice/Desktop/员工薪资明细表Q4_供应商.zip",
    "process_info": {"process_name": "Feishu.exe"},
    "window_info": {"window_title": "Feishu - 外部合作群 - 上传附件"}
  }
]
```

处理过程：

步骤	判断	结果
识别敏感源	file_open 事件中的文件名包含“薪资”，并且路径属于敏感文件列表或敏感关键词命中。	建立源文件记录：员工薪资明细表 Q4.xlsx。
推断派生文件	后续 compressed 事件生成 zip，文件名前缀与源文件相似，且时间顺序在源文件之后。	建立映射：员工薪资明细表 Q4.xlsx -> 员工薪资明细表 Q4_供应商.zip。
识别上传动作	file_upload 事件显示 zip 被 Feishu 上传，窗口标题包含“外部合作群”和“上传附件”。	生成 UploadEvent，应用类别为黑名单或高风险外部协作工具，触发告警。
跳过 VLM	日志已经能证明敏感文件派生物被上传，不需要视觉兜底。	节省一次 VLM 调用和最多 6 帧图片输入。
注入 Datalog	将打开、派生、上传转成 OpenFile、TransferFile、LeakFile。	ThreatDetector 输出完整泄露路径, 而不是孤立告警。

最终报告中可解释为：用户打开了薪资表，将其压缩成供应商 zip 包，并通过飞书外部群上传。系统能说明源文件、派生文件、上传应用、上传时间和映射链。

关键问题说明

关注点	说明
为什么先看日志	日志包含文件路径、进程名、事件类型和时间戳，是更便宜、更稳定的证据来源。文件上传、浏览器 temp 上传、云同步写入、HTTP POST、U 盘复制等场景可以直接通过日志形成证据链，无需让 VLM 反复看图。

为什么仍保留 VLM	有些风险只靠日志无法知道内容。例如剪贴板复制、截图、录屏、ChatGPT 输入框粘贴，日志只能说明动作发生了，不能说明复制/截图的是否是敏感内容。因此 VLM 被定位为视觉语义兜底。
如何避免误报	负例通过三道约束过滤：没有敏感上下文不触发 fallback；普通聊天、远距离 AI 噪声不触发 VLM；VLM 后处理会丢弃打开、阅读、浏览、滚动等低价值正常事件。
如何避免漏报	对真实外发方式做类别扩展，不再只依赖 file_upload。新增覆盖包括重命名/压缩/导出派生、浏览器临时文件、云同步目录、可移动盘、HTTP POST、屏幕共享、二维码、截图录屏。
benchmark 指标含义	当前 1.0 指标来自离线 fixture，不等同于生产准确率。它的作用是证明分支对已建模场景无回归，并提供后续接入真实标注集的评估框架。
token 消耗如何估算	离线 benchmark 统计“需要 VLM fallback 的样例数量”，并按 DLD_VLM_MAX_FRAMES=6 估算帧数。当前 fixture 下估算为 4 次 VLM fallback、24 帧。

关键术语解释

术语	含义
敏感源文件	被保护的数据源，例如薪资表、客户名单、合同、预算表、战略规划文档。检测系统首先要知道哪些文件或文件名模式属于敏感对象。
派生文件	由敏感源文件变化而来的文件，例如 客户合同.docx 另存为 合同扫描件.pdf，或 薪资表.xlsx 压缩成 薪资表.zip。派生文件名字可能完全不同，但内容仍来自敏感源。
文件映射链	用来说明“当前外发文件来自哪个原始敏感文件”的链路。例如 客户合同.docx -> 合同扫描件.pdf -> 邮件附件上传。
log-first	日志优先检测策略。先用文件路径、进程名、事件类型、窗口标题、时间戳等日志证据判断是否已经构成外发链路。能判断时不调用 VLM。
VLM fallback	视觉兜底策略。当日志只能说明动作可疑、但无法确认内容时，再调用 VLM 看画面。例如日志知道用户截图了，但不知道截图里是不是敏感预算表。
OCR 命中	从视频帧里识别到目标关键词或相似文本。OCR 命中是进入 VLM 的前置筛选，目的是避免把整段视频都送给模型。
UploadEvent	模块 3 输出的统一上传/外发事件结构，包含外发文件、原始敏感文件、应用名、黑白名单类别、操作类型、告警级别和映射链。
Datalog fact	ThreatDetector 使用的推理事实，例如 OpenFile、TransferFile、CrossProcessTransfer、LeakFile。这些 fact 可以组合成一条可解释泄露路径。
fixture	固定测试样例。每个 fixture 保存一组日志或 VLM 响应以及预期结果，用于回归测试和离线 benchmark。

分支提交时间线

提交	主题	主要内容
e2dbafe	perf: reduce API usage with log-first detection	建立日志优先检测链路；在确定性日志证据足够时跳过 VLM；新增敏感文件映射、上传事件构造与统计同步。
5b5d770	fix: handle upload-only and renamed sensitive files	修复只出现上传日志、以及敏感文件经过重命名/派生后上传时的映射缺失问题。
dee2003	Add token-aware VLM fallback scenarios	增加 VLM fallback gate 场景：AI 粘贴、截图、录屏、剪贴板复制等；限制 VLM 帧数并记录 token 优化元数据。
41d781b	Harden VLM event postprocessing for Qwen	强化 VLM JSON 解析、事件标准化、正常噪声过滤与去重，降低 Qwen 输出格式波动带来的漏检/误检。

eef0151	Add harder realistic accuracy fixtures	补充更难的真实样例，包括长文本粘贴、截图外发、前置说明 JSON、正常阅读负例等。
c9cea94	Add missed violation fixture coverage	构造当前项目原本识别不出的红队样例，覆盖字段别名、语义改写、屏幕共享、二维码、云同步、U 盘、HTTP POST 等。
a21efe1	Improve robust violation detection	面向类别而非用例修复：敏感概念组、风险动作词、字段别名、外部通道识别、延迟导出追踪。
87f8aa7	Add offline detection benchmark	新增 tools/benchmark_detection.py，输出 precision/recall/F1、case failures 与估算 VLM 调用压力。

模块级改动说明

模块 3 / RiskHunter: log-first 降成本与提召回

新增 3-RiskHunter/log_first_detector.py，让系统先使用日志完成确定性分析：

改造前，如果日志里出现了敏感文件和上传动作，但中间有重命名、压缩、浏览器临时文件等变化，系统更容易依赖后续 VLM 或模块间推断。这样的问题是成本高，而且日志中其实已经存在足够证据时没有被充分利用。改造后，模块 3 会先扫描日志，把敏感文件、派生文件和上传事件串成一条链。如果链路完整，就直接输出 UploadEvent。

- 识别敏感源文件：支持配置敏感文件、敏感文件名关键字、浏览器 temp upload metadata。
- 建立文件映射：从打开、创建、修改、重命名、复制、压缩、转换等日志推断派生链。
- 构造上传事件：当文件上传、HTTP POST、云同步目录写入、可移动盘写入等出现时，直接形成 UploadEvent。
- 黑白名单处理：黑名单应用触发告警，白名单应用记录但不告警；高置信外部通道如 cloud_sync、removable_media、network_upload 会提升风险。

这部分的意義是：日志能说明问题时不花 VLM token；日志不够时再交给 VLM 判断画面语义。

例子：用户打开 员工薪资明细表 Q4.xlsx，随后用 7-Zip 压缩成 员工薪资明细表 Q4_供应商.zip，再上传到飞书外部群。日志中已经有打开、压缩、上传三个事件，因此 log-first 可以直接判断为敏感数据外发，不需要让 VLM 去看视频画面。

模块 1 / FrameAnalyzer: VLM 输入与输出鲁棒化

在 1-FrameAnalyzer/agent.py 中完成两类增强。

模块 1 负责看录屏，但它不应该“无差别看所有画面”。一段录屏可能有成千上万帧，直接把大量图片送给 VLM 会带来高成本和高延迟。因此本轮把模块 1 拆成两步：先用 OCR 和关键词筛出可能相关的帧，再只把少量关键帧和上下文帧送给 VLM。

输入侧 token 控制：

- OCR 命中后才进入 VLM。
- DLD_VLM_MAX_FRAMES 控制最多发送帧数，默认 6。
- 采用“关键帧均匀采样 + 尾部上下文”的策略。
- 图片按 DLD_VLM_IMAGE_MAX_EDGE 缩放，JPEG quality 默认 65。
- 输出 vlm_optimization 元数据，便于审计 VLM 消耗。

输出侧鲁棒性：

- 支持 JSON fence、前置解释文本、裸数组等 Qwen 常见格式。
- 对字段别名做标准化：operation、action、file_name、source_file、target_file 等会合并到标准字段。
- 增加敏感概念组匹配：例如薪资/薪酬、账号/账户、预算/财务/董事会、合同/协议等。
- 增加风险动作词：粘贴、复制、发送、附件、上传、截图、录屏、屏幕共享、二维码、导出、编码等。
- 过滤正常打开、阅读、滚动浏览等低价值事件，避免把正常查看误当外发。

例子: VLM 返回 {"operation": "添加邮件附件", "file_name": "客户名单.xlsx"}。旧逻辑只认 operation_type 和 original_filename, 可能把事件丢掉; 新逻辑会把 operation 映射成 operation_type, 把 file_name 映射成 original_filename, 再进入统一过滤流程。

E2E / run_e2e: VLM fallback gate

run_e2e.py 中新增 _should_use_vlm_fallback:

这个门控函数解决的是“什么时候值得花 VLM token”的问题。它不是检测最终违规, 而是判断“日志证据不足时是否需要继续看画面”。判断条件包括三部分: 存在敏感上下文、附近有模糊外发动作、动作发生在合理时间窗口内。

- 如果 log-first 已经有确定上传事件, 直接返回结果, 跳过 VLM。
- 如果没有敏感上下文, 跳过 VLM。
- 如果敏感文件附近出现 AI、剪贴板、截图、录屏、上传、附件、云同步、U 盘、HTTP POST 等模糊信号, 触发 VLM。
- 可通过 DLD_VLM_FALLBACK_WINDOW_SEC 控制时间窗口, 默认 300 秒。

例子: 敏感 Word 文档打开后 18 秒, 浏览器窗口标题变成 ChatGPT, 并出现粘贴动作。日志能说明“可疑”, 但不能证明粘贴内容就是敏感文档, 此时 fallback gate 会触发 VLM。相反, 如果敏感文档打开 18 分钟后用户打开 ChatGPT 做旅行规划, 超过时间窗口, 会被当作噪声跳过。

实现细节一: VLM 输入成本控制

VideoFileOperationAgent._select_vlm_frames: 控制 VLM 图片输入数量

位置: 1-FrameAnalyzer/agent.py:71

核心逻辑:

```
max_frames = self._get_int_env("DLD_VLM_MAX_FRAMES", 6)
deduped = self._dedupe_frames(hit_frames)
key_frames = [item for item in deduped if item.get("type") == "key_event"]
context_frames = [item for item in deduped if item.get("type") != "key_event"]

selected_context = context_frames[-3:]
key_budget = max_frames - len(selected_context)
selected = self._pick_evenly(key_frames, key_budget) + selected_context
```

实现含义:

- 默认最多发 6 帧给 VLM, 避免视频长时段 OCR 多次命中导致 token 爆炸。
- key event 采用均匀采样, 而不是只取最前或最后, 避免漏掉中间真实动作。
- context frame 固定保留尾部最多 3 帧, 因为真实外发动作常常发生在“复制/选择文件”之后几秒。
- _dedupe_frames 按帧索引去重, 防止相同帧被重复送入 VLM。

具体例子:

输入帧: 10 个 key_event + 3 个 context, DLD_VLM_MAX_FRAMES=5

输出帧: 2 个均匀 key_event + 3 个尾部 context

收益: 从 13 帧降到 5 帧, 同时保留后续动作画面

测试覆盖: test_frame_analyzer_limits_vlm_frames_and_keeps_context 明确断言选出的最后三帧是 context 帧 [13, 18, 25]。

VideoFileOperationAgent._resize_for_vlm: 控制单张图片 token 体积

位置: 1-FrameAnalyzer/agent.py:98

核心逻辑:

```
max_edge = self._get_int_env("DLD_VLM_IMAGE_MAX_EDGE", 1280)
largest = max(height, width)
if largest <= max_edge:
    return frame
```

```
scale = max_edge / largest
return cv2.resize(frame, new_size, interpolation=cv2.INTER_AREA)
```

实现含义：

- 大图按最长边缩放到默认 1280。
- 不改变宽高比例。
- JPEG quality 在调用处使用 DLD_VLM_JPEG_QUALITY，默认 65。

对真实场景的影响：

- 录屏可能是 2K/4K，如果原图直接送 VLM，图片 token 和网络传输都会变高。
- 缩放到 1280 后，仍能保留文件名、附件 chip、聊天输入框等关键证据，但成本稳定。

实现细节二：VLM 响应解析与事件过滤

_parse_vlm_response_content：兼容 Qwen 常见 JSON 包装

位置：1-FrameAnalyzer/agent.py:202

核心逻辑：

```
cleaned = re.sub(r"<json-fence>|<fence>", "", str(content or ""), flags=re.IGNORECASE).strip()
decoder = json.JSONDecoder()
for pos, char in enumerate(cleaned):
    if char not in "[{":
        continue
    try:
        data, _ = decoder.raw_decode(cleaned[pos:])
        return data
    except json.JSONDecodeError:
        continue
```

解决的问题：

- Qwen 有时返回 Markdown JSON fence，例如 “下面是分析结果:” 后面包一段 json 代码块，代码块内才是 {"events": [...]}。
- 有时先说一句 “我无法完全确定，但可见证据如下:”，后面才接 JSON。
- 有时返回裸数组 [{...}]，而不是 {"events": [...]}。

改造后不是用简单 json.loads(resp.content)，而是从第一个可能的 { 或 [开始尝试 raw_decode。这能容忍模型前置说明，但仍要求最终有合法 JSON。

对应 fixture：

- qwen_json_fence_duplicates_and_unrelated_chatgpt_open
- qwen_refuses_with_text_before_valid_json
- qwen_bare_array_missing_operation_type_is_normalized

_normalize_vlm_event：把模型字段别名统一成标准事件

位置：1-FrameAnalyzer/agent.py:269

核心逻辑：

```
normalized.setdefault(
    "operation_type",
    event.get("operation") or event.get("action") or event.get("event_type") or "未知",
)
normalized.setdefault(
    "original_filename",
    event.get("original_file")
    or event.get("source_filename")
    or event.get("source_file")
    or event.get("file_name")
)
```

```

    or event.get("filename")
    or "未知",
)

```

解决的问题：

模型输出不会严格遵守字段名。例如我们期望：

```

{
  "operation_type": "邮件附件外发",
  "original_filename": "客户名单.xlsx"
}

```

但 Qwen 可能返回：

```

{
  "operation": "添加邮件附件",
  "file_name": "客户名单.xlsx"
}

```

以前 _event_text 不读取 operation 和 file_name，这个事件会在关键词过滤阶段被丢掉。现在标准化后：

```

{
  "operation_type": "添加邮件附件",
  "original_filename": "客户名单.xlsx"
}

```

对应红队样例：vlm_alias_fields_file_name_and_operation_are_ignored。修复前 kept=0，修复后 kept=1。

_event_text：扩大参与匹配的字段范围

位置：1-FrameAnalyzer/agent.py:166

新增读取字段包括：

```

"app", "application",
"operation", "action", "event_type",
"original_file", "source_filename", "source_file",
"file_name", "filename",
"target_filename", "target_file",
"summary", "evidence", "content", "ocr_text"

```

为什么重要：

- VLM 模型经常把证据写进 description、visual_evidence、detected_content。
- 不同模型或同一模型不同轮次会用不同字段名。
- 后处理如果只看固定字段，会把“模型已经识别出的违规”误删。

例子：

```

{
  "app_name": "Gmail",
  "operation": "添加邮件附件",
  "file_name": "客户名单.xlsx",
  "description": "文件被拖入邮件附件区并准备发送给外部收件人"
}

```

现在 _event_text 会把 app、operation、file_name、description 都纳入匹配文本，从而能命中“客户名单.xlsx + 附件/发送”的风险组合。

_keyword_concept_groups：用规则式语义近似补足文件名改写

位置：1-FrameAnalyzer/agent.py:112

核心概念组：

```

("薪资", "工资", "薪酬", "月薪", "salary", "payroll")
("客户", "客户名单", "customer", "client")

```



```
("预算", "成本", "财务", "董事会", "budget", "finance", "cost")
("账号", "账户", "银行", "收款", "account", "bank")
("合同", "协议", "contract", "agreement")
("战略", "规划", "路线图", "strategy", "roadmap")
```

解决的问题：

OCR/VLM 不一定复述精确文件名。例如：

关键词：薪资表.xlsx

VLM 描述：员工薪酬明细表、月薪数据、粘贴到 AI 对话框

以前短关键词模式只做精确 substring，薪资表.xlsx 不在事件文本里，直接漏检。现在逻辑是：

```
if event_has_risk_signal and shared_sensitive_concept(keyword_text, event_text):
    return True
```

也就是必须同时满足：

- 事件里有风险动作，如粘贴、上传、发送、屏幕共享、二维码；
- 关键词和事件文本共享同一类敏感概念，如薪资/薪酬。

这样比“只要出现敏感词就保留”更稳，能减少正常阅读误报。

`_risk_tokens`：补齐真实外发动作词

位置：1-FrameAnalyzer/agent.py:134

新增覆盖：

英文：paste, copy, clipboard, send, upload, attach, share, screenshot, screenrecord, record, qr, download, export

中文：粘贴、复制、剪贴板、发送、上传、附件、分享、外发、截图、录屏、屏幕共享、二维码、生成、下载、导出、转发、编码

具体解决的漏检：

- 屏幕共享：敏感预算没有文件名可见，但 VLM 看到“共享包含董事会预算数字的 Excel 画面”。
- 二维码：外发不是上传文件，而是把账号信息编码成二维码。
- 录屏/截图：不是传统文件上传，但仍可能把敏感画面带出本地可信环境。

红队样例：

- vlm_screen_share_contains_sensitive_sheet_but_filename_not_visible
- vlm_qr_code_exfiltration_has_target_data_but_no_risk_token

`_event_matches_keywords`：从单一精确匹配变成三层匹配

位置：1-FrameAnalyzer/agent.py:232

现在的匹配路径：

```
# 1. 长文本模式：模糊匹配长文本
if fuzz.partial_ratio(target_text, compact_event) >= 55:
    return True

# 2. 短关键词：精确 compact substring
if any(keyword in compact_event for keyword in compact_keywords):
    return True

# 3. 风险动作 + 敏感概念共享
if self._event_has_risk_signal(event) and self._shared_sensitive_concept(...):
    return True

# 4. 文件名形态模糊：去掉 . _ - / 空格后 partial_ratio >= 82
```

```
if fuzz.partial_ratio(filenameish_keyword, filenameish_event) >= 82:
    return True
```

为什么要多层：

- 精确匹配：保证常规场景高精度。
- 长文本模糊：处理复制粘贴长段落。
- 概念匹配：处理 VLM 语义改写。
- 文件名形态模糊：处理 OCR 丢符号、文件名分隔符变化。

负例保护：

- 在 `_is_low_value_normal_event` 中，只要事件是打开、阅读、浏览、滚动，且没有风险动作，就会被过滤。
- 例如 `qwen_target_file_normal_reading_is_dropped`：即使文件名命中，也因为只是阅读而被丢弃。

实现细节三：Log-first 确定性检测

`LogFirstDetector.analyze`：日志优先主循环

位置：3-RiskHunter/log_first_detector.py:229

主循环流程可以概括为四步：

```
for log in logs_by_time:
    path = normalize_path(log.get("file_path", ""))

    # 1. 如果 upload_detection.original_file 指向敏感源文件，直接建立映射
    detected_original = self._upload_detection_original_file(log)

    # 2. 如果当前 path 是敏感文件，登记 source_by_key
    if self._is_sensitive_path(path, log):
        source_by_key[file_key(path)] = {...}

    # 3. 尝试根据文件名、进程、时间窗口推断派生关系
    parent = self._find_parent_for_log(log, source_by_key, mappings)

    # 4. 如果当前日志是上传/外发通道，则构造 UploadEvent
    if is_upload_log(log):
        event = self._build_upload_event(log, source_by_key, mappings)
```

数据结构含义：

- `source_by_key`：记录“这个路径是否属于敏感链路，以及它的原始敏感源文件是谁”。
- `mappings`：记录“派生文件 -> 原始敏感文件”的映射。
- `operation_records`：记录打开、转换、上传等敏感操作，用于报告与 Datalog 注入。
- `upload_events`：最终检测到的上传/外发事件。

`_upload_detection_original_file`：解决浏览器 temp 上传

位置：3-RiskHunter/log_first_detector.py:355

真实浏览器上传经常出现这种日志：

```
{
  "event_type": "file_upload",
  "file_path": "C:/Users/alice/AppData/Local/Temp/chrome_upload_8f31.tmp",
  "upload_detection": {
    "original_file": "C:/Users/alice/Documents/BoardBudget_Confidential.xlsx",
    "temp_file": "C:/Users/alice/AppData/Local/Temp/chrome_upload_8f31.tmp"
  }
}
```

如果只看 `file_path`，上传的是 `.tmp`，看不出敏感性。现在逻辑会读取 `upload_detection.original_file`：

```
original = normalize_path(upload_detection.get("original_file", ""))
if self._is_sensitive_path(original, {"file_name": basename(original)}):
    return original
```

因此 temp 文件会被映射回真实敏感源文件 BoardBudget_Confidential.xlsx。

对应 fixture: browser_temp_upload_with_original_file_metadata。

_find_parent_for_log: 推断重命名、压缩、导出、派生文件

位置: 3-RiskHunter/log_first_detector.py:374

评分规则:

```
if current_stem.startswith(parent_stem) or parent_stem.startswith(current_stem):
    score += 4
if is_sensitive_name(current_stem) and is_sensitive_name(parent_stem):
    score += 2
if basename(parent_path).split(".")[0] in basename(path):
    score += 1
if event_type in DERIVE_TYPES and same_process and close_in_time:
    score += 3
if event_type in DERIVE_TYPES and same_process and export_window and
self._is_export_like_log(log):
    score += 3

return best_parent if best_score >= 3 else None
```

这不是单一规则，而是组合评分：

- 文件名相似：员工薪资明细表 Q4.xlsx -> 员工薪资明细表 Q4_供应商.zip。
- 敏感名称相似：两个文件名都含“合同/财务/客户/预算”等。
- 同进程短时间派生：Word 导出 PDF，Excel 导出 CSV/PDF。
- 同进程较长窗口导出：用户打开预算表几分钟后导出 notes.pdf，再上传。

具体例子：

```
13:00 打开 董事会预算.xlsx
13:08 Excel 创建 notes.pdf, 窗口标题 Export as PDF
13:09 Edge 上传 notes.pdf 到 Gmail
```

修复前: notes.pdf 和 董事会预算.xlsx 文件名不相似，且超过 120 秒，映射失败。

修复后: Export as PDF + same_process + 600 秒窗口 补足评分，建立映射，上传可被识别。

对应红队样例: delayed_generic_export_then_upload_loses_parent_mapping。

is_upload_log 与外部通道识别: 不只看 file_upload

位置: 3-RiskHunter/log_first_detector.py:196

外部通道分三类:

```
NETWORK_UPLOAD_TYPES = {
    "http_post", "http_put", "network_upload", "web_upload", "cloud_upload", "api_upload"
}

CLOUD_SYNC_MARKERS = ("dropbox", "onedrive", "google drive", "icloud drive", "云盘", "同步", ...)
REMOVABLE_MARKERS = ("removable", "usb", "thumb drive", "flash drive", "u盘", "可移动", ...)
NETWORK_UPLOAD_MARKERS = ("http://", "https://", " post ", " put ", "transfer.sh", "webdav")
```

is_upload_log 的最终判断:

```
if event_type in UPLOAD_TYPES:
    return True
if log.get("upload_detection", {}).get("is_upload"):
    return True
if window contains upload/send/attachment:
```

```
    return True
return bool(is_network_upload_log(log) or is_external_transfer_log(log))
```

注意：云同步和 U 盘不是只要出现就算外发，代码里通过 `is_external_transfer_log` 收紧：

```
write_like_types = DERIVE_TYPES | UPLOAD_TYPES | {"file_write", "file_create", "sync_upload"}
return event_type in write_like_types and bool(external_destination_reason(log))
```

也就是说：

- 打开 Dropbox 里的文件：不算外发。
- 把敏感文件复制到 Dropbox 同步目录：算外发。
- 打开 U 盘文件：不一定算外发。
- 把本地敏感文件复制到 E:/... 且进程是 `explorer.exe`：算可移动介质外发。

对应红队样例：

- `cloud_sync_folder_copy_is_not_treated_as_exfiltration` -> `operation_type=cloud_sync`
- `usb_copy_is_only_a_derived_file_not_a_leak` -> `operation_type=removable_media`
- `http_post_with_sensitive_file_path_not_classified_as_upload` -> `operation_type=network_upload`

`_build_upload_event`：统一输出可审计事件

位置：3-RiskHunter/log_first_detector.py:421

关键字段：

```
UploadEvent(
    file_path=path,
    file_name=basename(path),
    original_file=source.get("original_file") or path,
    upload_content=path,
    upload_content_mapping_link=self._mapping_chain(path, mappings),
    app_name=app_name,
    app_category=app_category,
    behavior_category="data_exfiltration",
    operation_type=operation_type,
    should_alert=should_alert,
    extra_info={
        "source": "log_first",
        "confidence": 0.95,
        "external_channel": operation_type,
    },
)
```

告警策略：

```
high_confidence_external = operation_type != "file_upload"
should_alert = app_category == "blacklist" or (high_confidence_external and app_category != "whitelist")
```

解释：

- 黑名单应用上传：critical。
- 白名单应用上传：记录但不告警。
- 云同步/U 盘/HTTP POST 这类高置信外部通道，即使不是传统黑名单应用，也会告警或 warning。
- `upload_content_mapping_link` 会保留派生链，例如 `客户合同-华东.docx` -> `合同扫描件 0628.pdf`。

实现细节四：E2E 门控、证据链与评估

`_should_use_vlm_fallback`：VLM 触发门控

位置：run_e2e.py:237

核心判断：

```

if sensitive_count <= 0:
    return False, reason="no_sensitive_log_context"

for log in logs:
    has_ai_context = _contains_any(text, AI_CONTEXT_TOKENS)
    has_ambiguous_exfil = _contains_any(text, AMBIGUOUS_EXFIL_TOKENS)
    near_sensitive = any(abs(timestamp_ms - sensitive_ms) <= window_ms)
    if near_sensitive and (has_ai_context or has_ambiguous_exfil):
        decision["candidate_events"].append(...)

if candidate_events:
    return True, decision="run"

```

触发词分组：

- AI 场景：ChatGPT、Claude、Gemini、DeepSeek、Kimi、豆包、通义、Cursor、Copilot 等。
- 模糊外发：clipboard、paste、copy、send、upload、attach、screenshot、recording、dropbox、onedrive、usb、http、transfer.sh 等。
- 中文动作：粘贴、复制、发送、上传、附件、截图、录屏、同步、云盘、U 盘、可移动。

几个例子：

```

敏感文件 10:00 打开, 10:00:18 ChatGPT 粘贴 -> run
敏感文件 12:00 打开, 12:00:31 普通飞书 daily standup -> skip
敏感文件 13:00 打开, 13:18 ChatGPT 旅行规划 -> skip (超过默认 300 秒窗口)
公开文件粘贴到 AI, 没有敏感上下文 -> skip

```

这就是“日志先行，视觉兜底”的实际开关。

run_module3_pipeline：主流程如何使用 log-first 与 fallback

位置：run_e2e.py:395 附近

主流程关键行为：

```

log_first_result = LogFirstDetector(...).analyze(logs_data)
upload_count = len(log_first_result.get("upload_events", []))

if upload_count > 0:
    return log_first_result # 证据足够，跳过 VLM

should_run_vlm, vlm_fallback_meta = _should_use_vlm_fallback(logs_data, log_first_result)
if not should_run_vlm:
    return log_first_result # 既无确定上传，也无必要视觉兜底

# 否则进入原 LangGraph + Module2 + Module1 视觉分析链路

```

这段代码直接决定 token 成本：

- upload_count > 0 是最省钱路径。
- should_run_vlm=False 是负例省钱路径。
- 只有“没有确定上传，但存在可疑视觉语义”才继续跑 VLM。

Datalog 注入与证据链连通

位置：run_e2e.py 中 _inject_connected_facts_from_module3

虽然本轮重点是模块 1/3，但 E2E 报告最终要落到 ThreatDetector 的证据链，所以还补强了从模块 3 结果到 Datalog fact 的衔接。

典型注入链：

```

OpenFile(wps.exe, 员工薪资明细表Q4.xlsx)
TransferFile(wps.exe, 员工薪资明细表Q4.xlsx -> 员工薪资明细表Q4_part1.xlsx)
CrossProcessTransfer(wps.exe -> msedge.exe, 员工薪资明细表Q4_part1.xlsx)
LeakFile(msedge.exe, 员工薪资明细表Q4_part1.xlsx)

```


测试输出中可见：

```
+ OpenFile(wps.exe, 员工薪资明细表Q4.xlsx)
+ TransferFile(wps.exe, 员工薪资明细表Q4.xlsx -> 员工薪资明细表Q4_part1.xlsx)
+ LeakFile(msedge.exe, 员工薪资明细表Q4_part1.xlsx)
+ CrossProcessTransfer(wps.exe -> msedge.exe)
发现 1 条泄露路径
```

这保证检测结果不仅是“发现上传”，还可以解释“源文件如何变成上传文件、经过哪个进程外发”。

tools/benchmark_detection.py: 离线评估器

位置：tools/benchmark_detection.py

评估器不调用真实 VLM，只评估后处理和日志分流逻辑。

核心数据结构：

```
@dataclass
class BinaryMetrics:
    tp: int = 0
    fp: int = 0
    tn: int = 0
    fn: int = 0

    @property
    def precision(self): ...
    @property
    def recall(self): ...
    @property
    def f1(self): ...
```

VLM fixture 评估：

```
parsed = agent._parse_vlm_response_content(case["response"])
raw_events = agent._coerce_event_list(parsed)
final_events, meta = agent._filter_vlm_events(raw_events, case["keywords"])
```

日志 fixture 评估：

```
result = LogFirstDetector(...).analyze(case["logs"])
should_run_vlm, fallback_meta = run_e2e._should_use_vlm_fallback(case["logs"], result)
predicted_triage_positive = upload_count > 0 or should_run_vlm
```

VLM 压力估算：

```
if item["upload_events"] == 0 and item["vlm_decision"] == "run":
    result.estimated_vlm_calls += 1
    result.estimated_vlm_frames += _vlm_frame_budget()
```

也就是说，benchmark 统计的是“还需要视觉兜底的样例数量”，而不是把所有样例都算成 VLM 调用。

典型样例前后对比

样例	修复前	修复后	对应代码
字段别名	VLM 返回 operation / file_name, 后处理只看 operation_type / original_filename, 事件被丢弃。	_normalize_vlm_event 将别名映射到标准字段, kept=1。	_normalize_vlm_event, _event_text
薪资/薪酬改写	关键词是“薪资表.xlsx”, VLM 写“薪酬明细/月薪数据”, 短关键词精确匹配失败。	敏感概念组判断薪资/薪酬同类, 且事件有“粘贴外发”, 保留。	_keyword_concept_groups, _event_matches_keywords

屏幕共享预算	画面没有精确文件名，只有“董事会预算数字和成本明细”，事件被关键词过滤。	预算/成本/财务概念 + 屏幕共享风险动作命中，保留。	_risk_tokens, _shared_sensitive_con
二维码外发	不是传统上传/发送，风险词列表不覆盖二维码编码。	qr、二维码、编码、生成 纳入风险动作。	_risk_tokens
云同步	copied 到 Dropbox 目录只被当成普通派生文件，不产出外发事件。	external_destination_reason 标记 cloud_sync，构造 UploadEvent。	is_external_transfer_log, _build_upload_event
U 盘拷贝	复制到 E:/... 不属于 file_upload，原逻辑跳过。	E:/ + explorer/removable 标记为 removable_media。	external_destination_reason
HTTP POST	http_post 不在上传事件类型中。	NETWORK_UPLOAD_TYPES 覆盖 http_post/http_put/api_upload。	is_network_upload_log
延迟导出	打开预算表 8 分钟后导出 notes.pdf，超过 120 秒且文件名不相似，映射断裂。	export_window=600s + Export as PDF 判定导出派生，上传可追溯。	_find_parent_for_log, _is_export_like_log

抽象能力沉淀

本轮修复没有按 case id 或固定文件名写死，而是抽象成几类可复用能力：

能力	抽象规则	覆盖面
字段别名	operation/action/file_name/source_file/target_file 等统一进入标准事件	不同 VLM 输出格式
敏感概念	薪资、客户、预算、账号、合同、战略、并购等概念组	文件名语义改写
风险动作	粘贴、上传、截图、录屏、屏幕共享、二维码、导出、编码等	非传统上传外发
派生映射	文件名相似、同进程、时间窗口、导出上下文综合评分	改名、压缩、导出
外部通道	网络 POST、云同步、可移动介质写入	日志不叫 file_upload 的外发
VLM 门控	敏感上下文 + 模糊动作 + 时间窗口	控制 token 与误报

这意味着后续新增类似场景时，不需要为每个样例单独补丁；只需要扩展概念组、通道 marker 或风险动作词。

场景覆盖

类别	已覆盖场景	收益
确定性日志	压缩后上传、改名后上传、浏览器 temp 上传、HTTP POST、云同步、U 盘拷贝、延迟导出再上传	减少 VLM 调用；增强无画面时的召回
需 VLM 判断	敏感文档后粘贴到 AI、截图后粘贴到微信、复制到剪贴板、录屏控件出现	日志无法知道内容时保留视觉判断
VLM 后处理	Qwen JSON fence、裸数组、字段缺失、重复事件、正常阅读噪声、语义改写、二维码外发	减少模型输出格式导致的漏检
负例	普通聊天、远距离 ChatGPT 噪声、公开资料上传、仅打开/阅读敏感文件	控制误报

验证结果

本轮验证分两层：单元/回归测试与离线 benchmark。

回归测试：

```
python -m unittest tests.test_e2e_regressions
Ran 17 tests in 14.232s
OK
```

离线 benchmark：

```
python tools/benchmark_detection.py
vlm_postprocess      precision=1.0000 recall=1.0000 f1=1.0000 accuracy=1.0000
log_triage           precision=1.0000 recall=1.0000 f1=1.0000 accuracy=1.0000
deterministic_resolution precision=1.0000 recall=1.0000 f1=1.0000 accuracy=1.0000
estimated_vlm_calls=4
estimated_vlm_frames=24
case_failures=0
```

说明：以上指标来自项目 fixture 与红队样例的离线评估，不等同于真实生产准确率；它证明当前规则对已建模场景无回归，并为后续真实标注集提供了量化入口。

Token 消耗分析

本轮优化的核心不是“完全不用 VLM”，而是让 VLM 只出现在真正需要视觉语义的环节。

- 最省 token 的路径：日志证据完整，如文件上传、重命名后上传、浏览器 temp metadata、云同步、U 盘、HTTP POST。此时 log-first 可直接给出事件。
- 中等 token 的路径：日志只有敏感上下文和模糊动作，如复制、粘贴、截图、录屏、AI 窗口。此时才触发 VLM fallback。
- VLM 输入上限：默认每次最多 6 帧，并压缩图片尺寸与 JPEG 质量。
- 当前 fixture 下估算 VLM 压力：4 次 fallback、24 帧。

因此，现在的成本模型更接近“日志先行，视觉兜底”。真实环境中 VLM 成本取决于模糊事件密度，例如频繁截图、复制、AI 窗口切换会增加 fallback 次数。

当前状态与风险

当前分支状态：

- 本地分支 h 已推送到 origin/h。
- 最新提交：87f8aa7 Add offline detection benchmark。
- 工作区在推送后保持干净。

仍需注意的风险：

- 离线 fixture 指标是回归指标，不代表真实生产准确率。
- 敏感概念组是规则式语义近似，真实语料中可能出现新的同义词或行业术语。
- 云同步/U 盘/HTTP POST 属于高风险外部通道，但企业环境中可能存在授权同步盘，需要结合组织白名单策略。
- VLM 的真实 token 价格依赖供应商图片计费规则，目前 benchmark 只估算调用次数和帧数。

下一阶段工作规划

下一阶段计划建设 60 条左右真实标注集：

- 30 条正例：覆盖文件上传、粘贴外发、截图外发、录屏、云同步、U 盘、HTTP POST、AI 对话。
- 30 条负例：覆盖正常打开、内部协作、公开资料处理、远距离噪声、白名单系统。
- 每条记录标注：是否违规、是否应告警、是否需要 VLM、预期外发链路、误报/漏报原因。

真实样例接入方式为：沿用 tools/benchmark_detection.py 的输入口径，将标注样例转成 fixture 后，即可持续输出 precision、recall、F1、fallback 次数和失败样例列表。

结论

本轮工作完成了从纯视觉依赖到“日志优先、视觉兜底”的检测链路升级。系统在日志证据充分时可直接识别敏感文件外发，从而减少 VLM 调用；在日志证据不足但存在 AI、剪贴板、截图、录屏等模糊信号时，再

触发 VLM 进行画面语义判断。同时，分支补齐了 Qwen 返回格式不稳定、字段别名、文件名语义改写、云同步、U 盘、HTTP POST 等真实场景。当前基于 28 个离线 fixture 样例的 benchmark 无失败，并已形成可复用评估脚本，可继续接入真实标注集评估生产准确率。